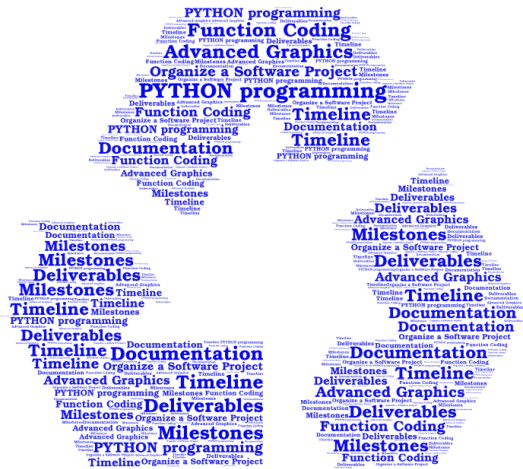




Python Programming: Work Package 4 – System

Lecture: Modelling an Simulation

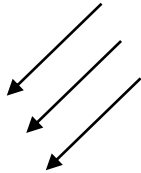
This is a compulsory elective module
of
**Technology of Circular
Economy**
(Master of Engineering)

Dr. Uwe Reimer
Senior Scientist

Overview of the project

Design a PV system which collects energy from the sun and stores it in a battery. An electric scooter shall be charged with this energy for commuting to work (daily drive cycle).

Results shall be presented for the city of Emden and second location in a different country.



Project Plan

		week						
Work package		1	2	3	4	5	6	
	Planning	*	M 1					
WP 1	Scooter							
WP 2	Solar panel							
WP 3	Battery					*	M 2	
WP 4	System Design							
WP 5	Calculate results						*	M 3
	Write report							
List of Deliverables				List of Milestones				
D 1	Software requirements	M 1		Plan ready				
D 2	Scooter model	M 2		All models ready				
D 3	PV model	M 3		Results ready				
D 4	Battery model							
D 5	System model							

These are the steps for the system development:

- 1) Review, which models we have created
- 2) Choose system configuration (central system controller vs linear)
- 3) Sort out system design: which steps should be modeled (what is first, second, ...)
- 4) Initialize model, create a loop over 365 days (1 year)
- 5) Add weekly cycle for scooter model
- 6) Test solar, scooter and battery model integration
- 7) Creating first results (one year)
- 8) Add control for PV system size
- 9) Calculate values per month
- 10) Create overview

Work package 4 / WP 4: System Software Design

Input:

1. Generation of energy per day: **M**
2. Distance driving: **C / W**
3. Power distribution to the scooter (Discharging): **C**
4. State of charge (%): **C / W**
5. Speed: **C / W**

Output:

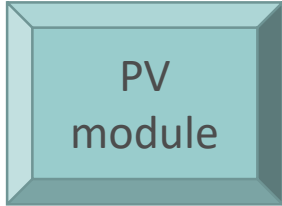
1. Charging and discharging rate : **C**
2. Distance driving: **C / W**

Assumptions:

1. Linear / System Controller : **M**
2. Energy losses: **W**

This time, it is not helpful to start from here.

We already have 3 models. Let us look at these models again and see, how we can put it all together.



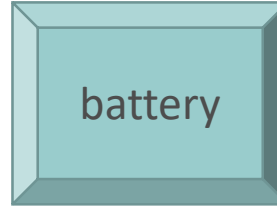
Initialization:

- readyear(filename)

Function:

- getday(d)

-> returns energy per day



Initialization:

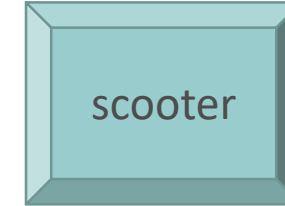
- none - 'empty battery'

Function:

- charge(p)
- discharge(p)

Variables:

- max. capacity
- energy stored
- charged state (%)



Initialization:

- readweek(list of files)

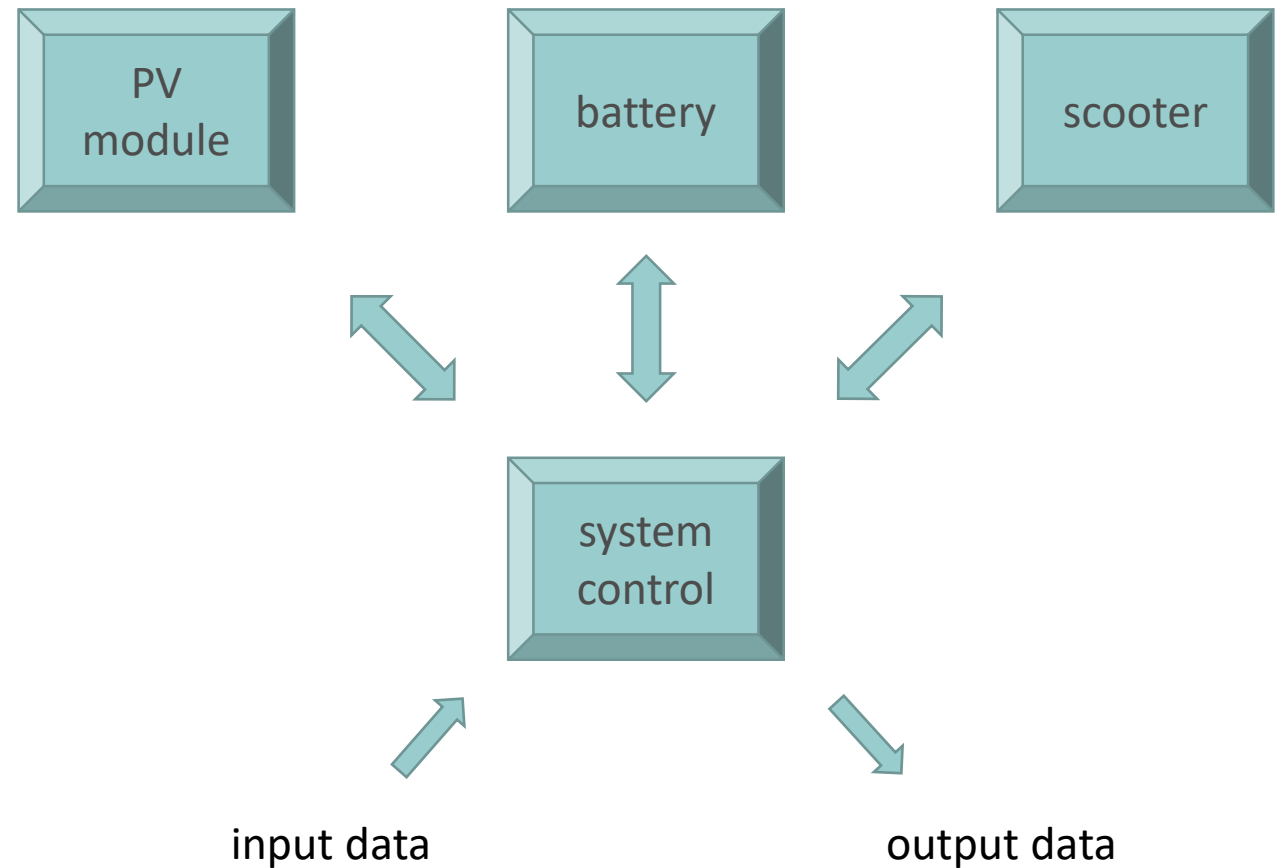
Function:

- getday(d)

-> returns energy per day and distance per day

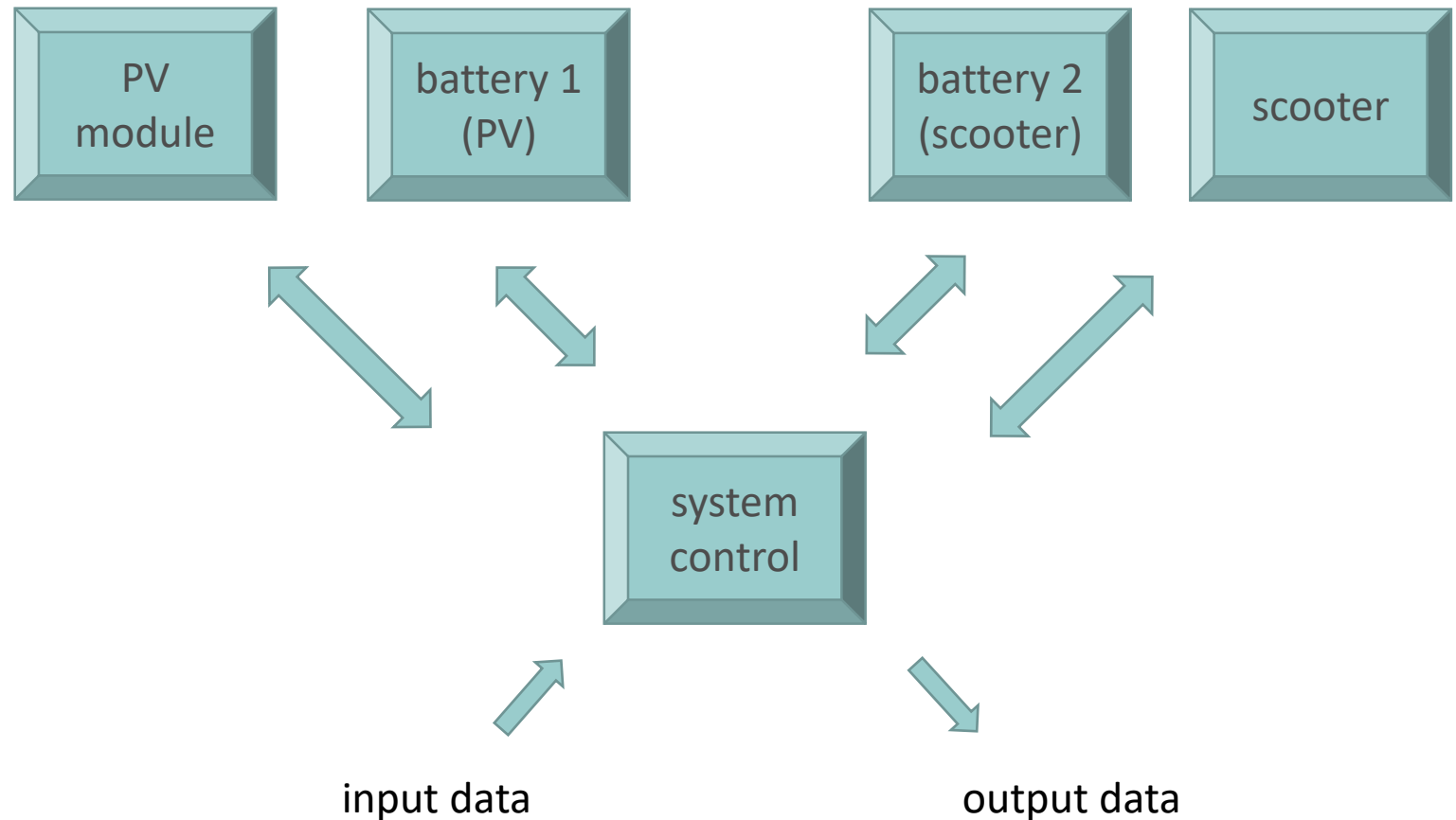
System controller configuration

Actually, we have two batteries.



System controller configuration

Let us start by collecting all the data that we need.



Step 1: Load all models

You can download this starting package from the Moodle course.

For the beginning, we need the following files:

- `batteryscooter.py` (This is a copy of the battery model.)
- `batterysolar.py` (This is a copy of the battery model.)
- `scooter.py`
- `solar.py`
- `sys01.py` (The start for our system model.)

Data files:

- `Emden-2023.csv`
- `Kochi-2023.csv`
- `day1.dat ... day7.dat`

Step 1: Load all models – sys01.py

This is how these files are loaded.

```
6 import batteriesolar as batsol    # model for battery of
7 import batteryscooter as batsco  # model for battery of
8 import solar                    # model for PV panel
9 import scooter                  # model for scooter
10
11 filescooter = [ 'day1.dat', 'day2.dat', 'day3.dat', 'da
12 filesolar = ['Kochi-2023.csv', 'Emden-2023.csv']
```

Initialization

We will start with an empty battery for solar panel but with a fully charged one for the scooter. Otherwise, we could not drive the first day (would need special considerations).

```
14 # initialize models, read files
15 solar.readyear(filesolar[0]) # we choose data
16 scooter.readweek(filescooter) # read driving c
17 batsco.capa = batsco.capamax # we start with
18 batsco.state = 100.0
```

Step 2: Sort out the single steps

What should happen in which order?

With our models we can perform the following actions.

solar

get energy per day

battery sol

charge()
discharge()

scooter

get energy per day
get distance per day

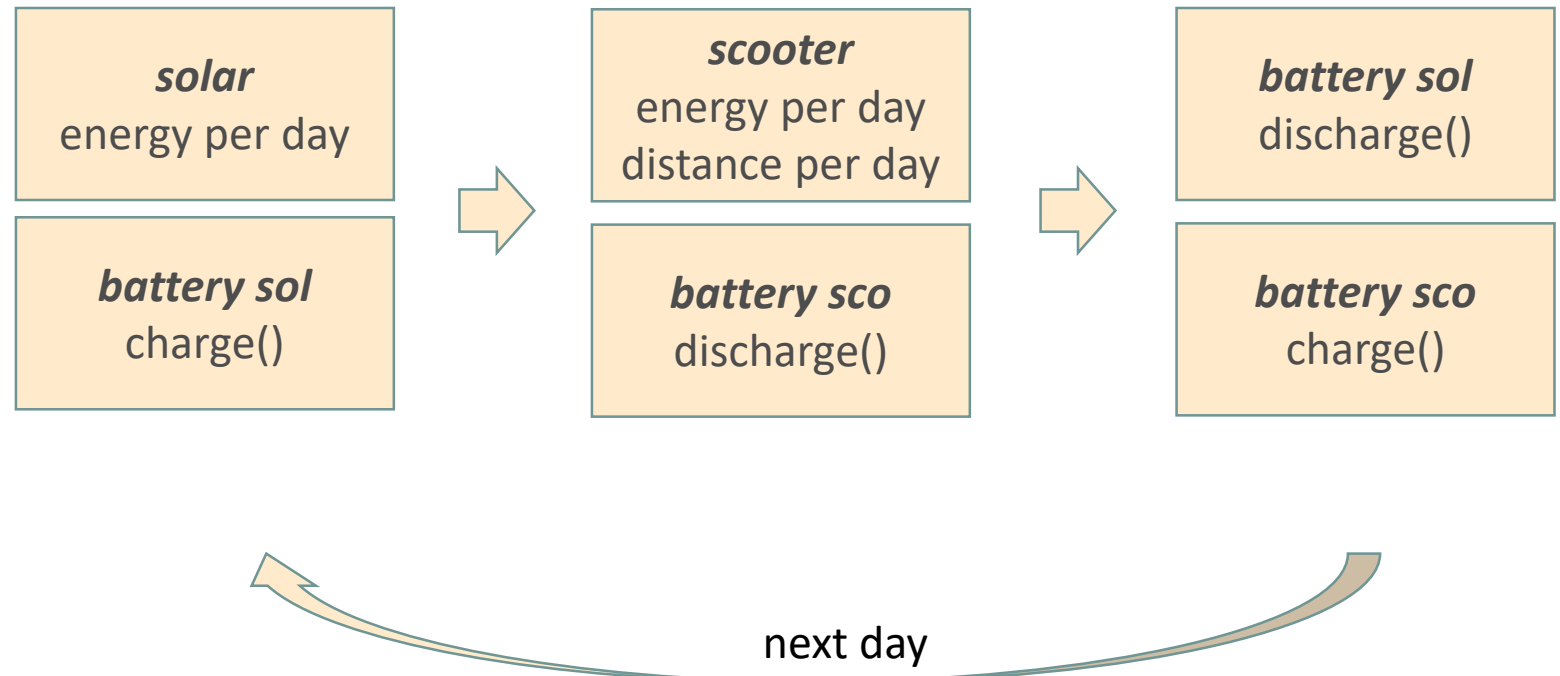
battery sco

charge()
discharge()

Step 2: Sort out the single steps

The day starts with sunshine. Energy is stored in the battery (solar).
In the evening the scooter returns home and has to be charged.
This is the procedure for each day of the year.

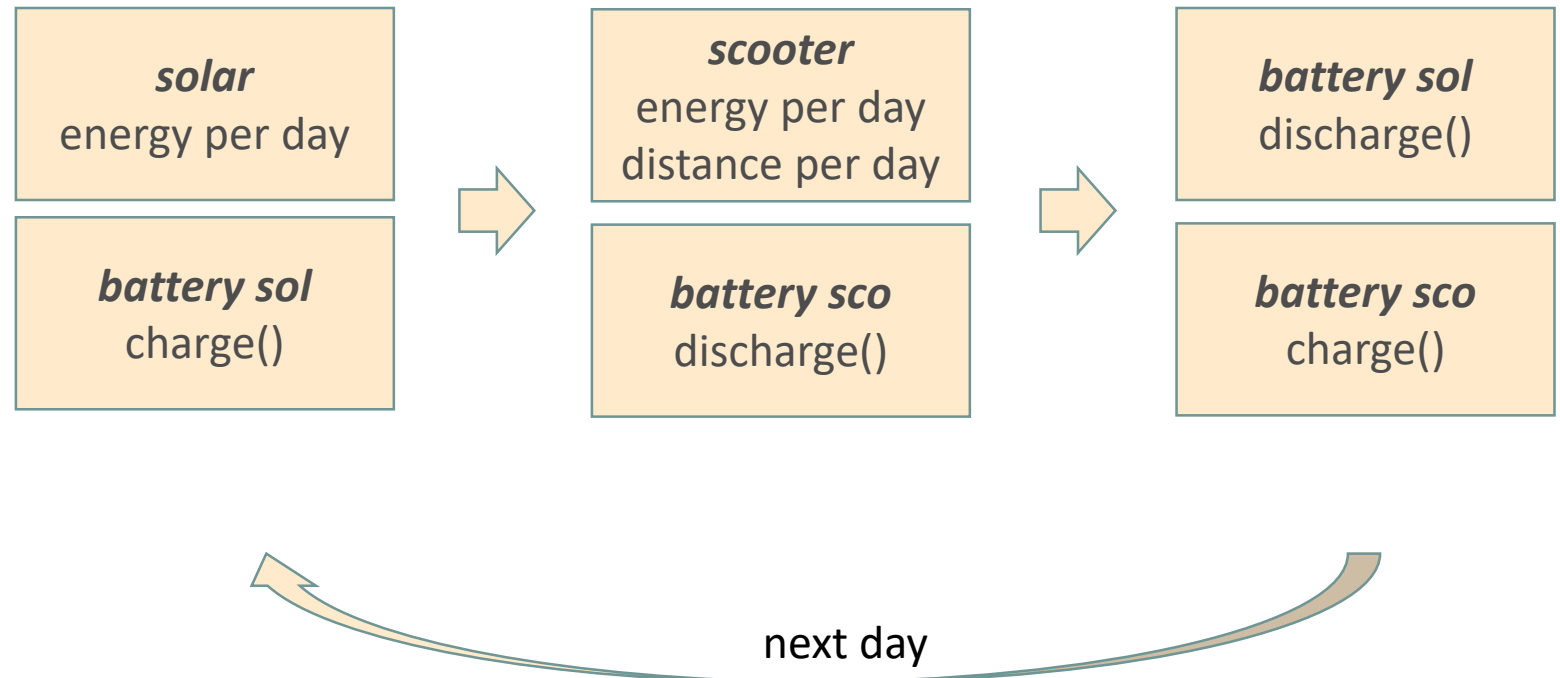
What is the essential information,
that we need here?
Remember the purpose of
the system model.



(Which variables need to be defined
for the system?)

Step 2: Sort out the single steps

The day starts with sunshine. Energy is stored in the battery (solar).
In the evening the scooter returns home and has to be charged.
This is the procedure for each day of the year.



Which variables need to be defined for the system?

- PV production per day
- Charge to scooter
- State of battery solar (min value)
- State battery scooter (min value)

Add the following code to our program.

```
23  ### System data collection ###  
24  pvpro = []          # PV production per day  
25  chargesco = []     # charge battery scooter  
26  statepv = []       # state battery solar (min value)  
27  statesco = []      # state battery scooter (min value)  
28
```

Everything is calculated per day.

In order to cover the period for one year, we need to define a loop over 365 days.

All calculations will happen inside this loop.

```
29  ### START to read the data for one year ###
30  d = 0
31  while d < 365:
32      d = d + 1    # day of the year
33
34      sol = solar.getday(d)    # solar power ir
35      pvpro.append(sol)        # store PV produ
36
37  ### END to read the data for one year ###
38
39  # check if OK this should be 365
40  print( len(pvpro) )
```


These are the steps for the system development:

- 1) Review, which models we have created 
- 2) Choose system configuration (central system controller vs linear) 
- 3) Sort out system design: which steps should be modeled (what is first, second, ...) 
- 4) Initialize model, create a loop over 365 days (1 year) 
- 5) Add weekly cycle for scooter model
- 6) Test solar, scooter and battery model integration
- 7) Creating first results (one year)
- 8) Add control for PV system size
- 9) Calculate values per month
- 10) Create overview

Now, the scooter is using up energy from it's own battery.
For the driving cycle, we need to introduce another variable = the day of the week.

```
29  ### START to read the data for one year ###
30  d = 0
31  w = 0
32  while d < 365:
33      d = d + 1    # day of the year
34      w = w + 1    # day of the week
35
```

```
36 sol = solar.getday(d)      # solar power
37 pvpro.append(sol)         # store PV prc
38
39 # first, we charge solar battery
40 a = batsol.charge(sol)
41
42 # second, the scooter is driving with
43 p = scooter.getday(w)
44 batsco.discharge(p[0])
45 a = batsco.state
46 statesco.append(a)        # store min state
47
48 if w > 6:                  # one week has 0..6 da
49     w = 0
50
51 ### END to read the data for one year ###
52
53 # check if OK this should be 365
54 print( len(pvpro) )
55 print( len(statesco) )
```

Charging solar battery

Discharging scooter

Now, the battery of the scooter needs to be charged. Remember, what was discussed for the battery model.

-> for the function of discharging: You cannot discharge more than what is stored.

Check, if you want to discharge to much.

- YES: Discharge all there is / battery will be empty.
- NO: You can safely discharge the requested amount.

-> for the function of charging: You cannot exceed maximum capacity.

Check, if you want to charge to much.

- YES: Charge only the amount until you reach maximum capacity.
- NO: You can safely charge the requested amount.

Charging the scooter goes together with discharging the solar battery.
Which steps need to be considered?

Charging the scooter goes together with discharging the solar battery.
Which steps need to be considered?

- How much energy is available?
- How much energy is needed?
- If we have more than needed:
 - Fully charge scooter.
- Otherwise:
 - Charge scooter with what we have.
(Solar battery will be empty.)

```
47
48 # now, scooter needs to be charged
49 # first, we check if there is enough solar power stored
50 a = batsol.capa # this is how much
51 x = batsco.capamax - batsco.capa # this is how much
52 if a >= x:
53     batsol.discharge(x)
54     batsco.charge(x)
55     chargesco.append(x) # store battery charge sco
56 else:
57     batsol.discharge(a)
58     batsco.charge(a)
59     chargesco.append(a) # store battery charge
60
61 a = batsol.state
62 statepv.append(a) # store min state of battery solar
63
```

At the end of our program, we test if all lists have the same size, i.e. results are stored.

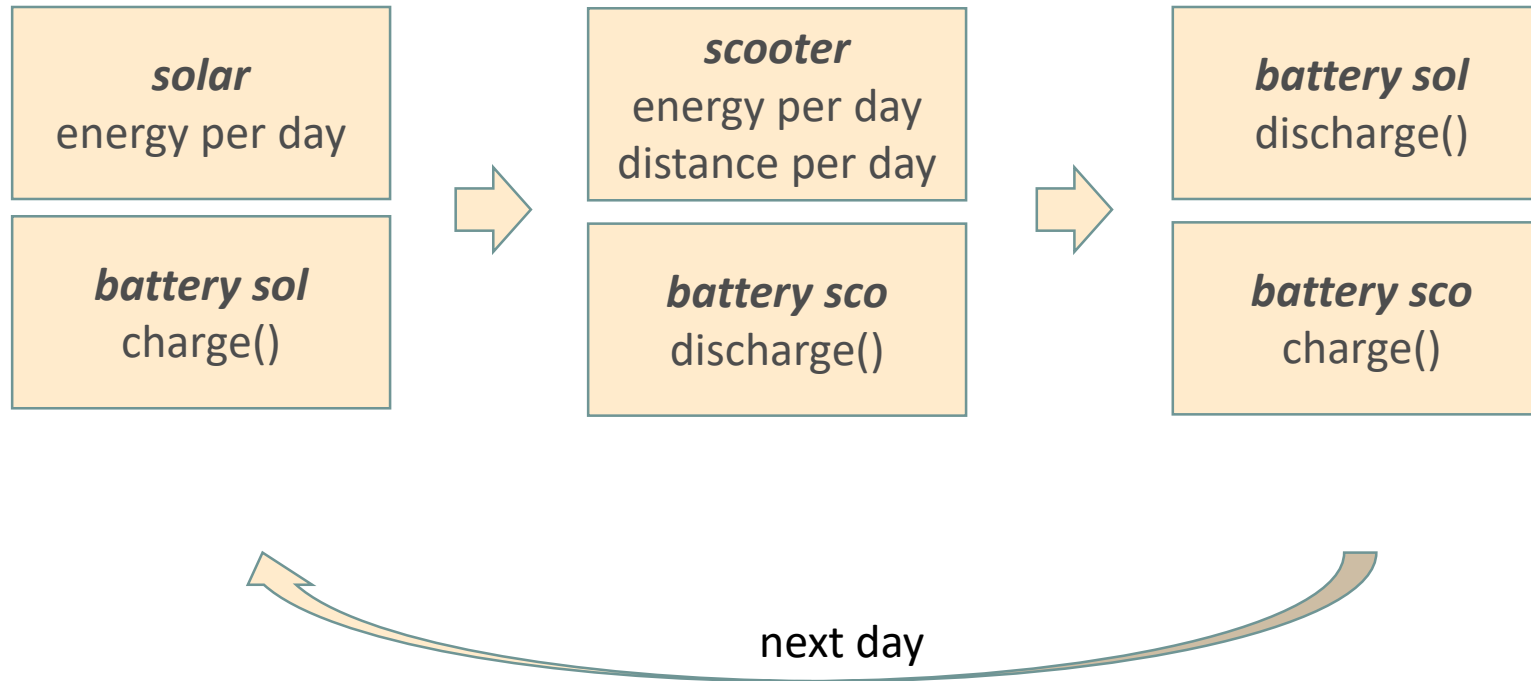
```
69 # check if OK this should be 365
70 print( len(pvpro) )
71 print( len(statesco) )
72 print( len(chargesco) )
73 print( len(statepv) )
```

This is a very simple way of testing, but it helps to find errors early in the process of programming.

These are the steps for the system development:

- 1) Review, which models we have created 
- 2) Choose system configuration (central system controller vs linear) 
- 3) Sort out system design: which steps should be modeled (what is first, second, ...) 
- 4) Initialize model, create a loop over 365 days (1 year) 
- 5) Add weekly cycle for scooter model 
- 6) Test solar, scooter and battery model integration 
- 7) Creating first results (one year)
- 8) Add control for PV system size
- 9) Calculate values per month
- 10) Create overview

We have successfully calculated all results.



What should be our output?

We have the following properties for each day of the year.

- PV production per day
- Charge to scooter
- State of battery solar (min value)
- State battery scooter (min value)

Let's take the first two points (PV production and charge to scooter) and create a plot for one year.

First results – sys05.py

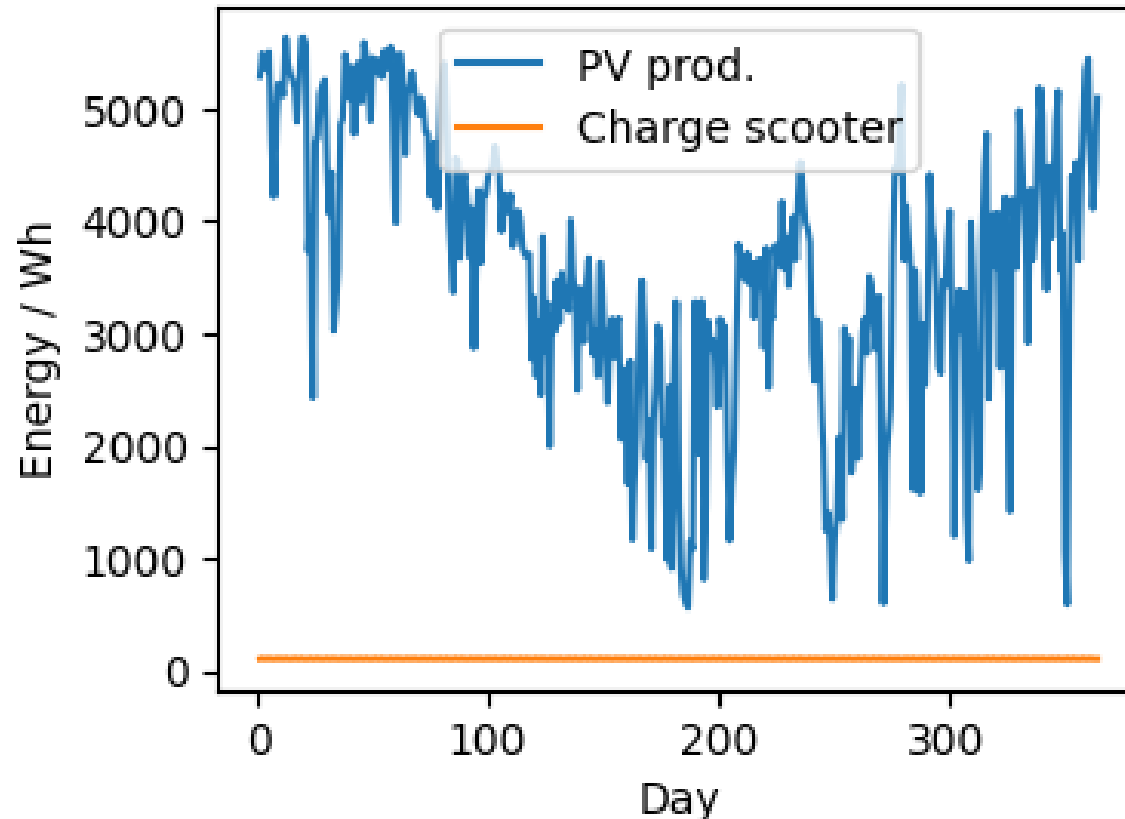
```
69  ### create example output
70  x = range(1,366,1) # number of days from 1..
71
72  pngfile = 'Energy.png'      # file name for
73
74  fwidth = 10                 # figure width i
75  fwidth = fwidth / 2.54      # conversion inc
76  fheight = fwidth * 3/4      # using a ratio
77  plt.subplots( figsize=( fwidth,fheight ) )
78
79  plt.plot(x, pvpro, label='PV prod.') # li
80  plt.plot(x, chargesco, label='Charge scooter
81
82  plt.xlabel('Day')           # label x-axis
83  plt.ylabel('Energy / Wh')   # label y-axis
84
85  plt.legend()
86  plt.tight_layout()          # optimize place
87  plt.savefig(pngfile)        # create file fc
```

← create a list with number of days

} plot size and name for file

} add two plots

} name for x- and y-axis



These are the results for the city of Kochi.

You can see that PV production is much higher than energy consumption of the scooter, because the model uses a fixed value for 1 kW peak power of the PV system.

We would like to adjust the size of the PV system.

-> We need to add a variable.

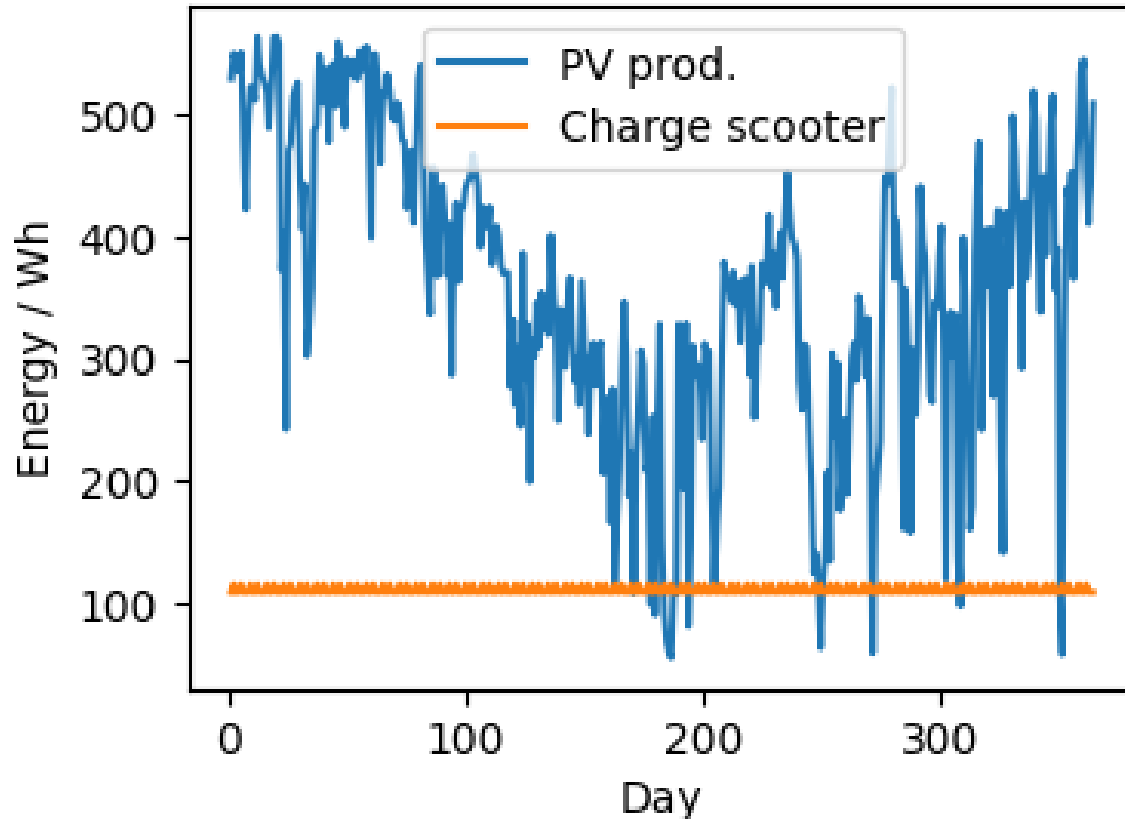
Adjusting PV size – sys06.py

Set the size of PV in W.

```
29  ### Control parameters ###
30  sizepv = 100.0 / 1000.0          # size of PV
31
32  ### START to read the data for one year ###
33  d = 0
34  w = 0
35  while d < 365:
36      d = d + 1      # day of the year
37      w = w + 1      # day of the week
38
39      sol = solar.getday(d)      # solar power i
40      sol = sol * sizepv        # adjusting siz
41      pvpro.append(sol)         # store PV proc
```

Scale the size of PV

Adjusting PV size – sys06.py



Now, the PV energy production and the consumption by the scooter are roughly in the same range. We could already start to work with this model.

Although, it would be nice to see more detail. Let's show the results for each month. Remember, we did something similar in a past lecture for calculating the PV production for each month. With that, we could identify the effect of the different seasons of the year.

These are the steps for the system development:

- 1) Review, which models we have created 
- 2) Choose system configuration (central system controller vs linear) 
- 3) Sort out system design: which steps should be modeled (what is first, second, ...) 
- 4) Initialize model, create a loop over 365 days (1 year) 
- 5) Add weekly cycle for scooter model 
- 6) Test solar, scooter and battery model integration 
- 7) Creating first results (one year) 
- 8) Add control for PV system size 
- 9) Calculate values per month
- 10) Create overview

First, we need to prepare the list
of the days for the start of each month.

The resulting list is:

0
31
59
90
120
151
181
212
243
273
304
334
365

Values per month – sys07.py

```
71  ### END to read the data for one year ###
72
73  ### create output for one month
74  # These are the number of days per month
75  days = []
76  days.append(0)    # This serves as starting point
77  days.append(31)   # January 31 days
78  days.append(28)   # Febr 28 days
79  days.append(31)   # Mar 31 days
80  days.append(30)   # Apr 30 days
81  days.append(31)   # May 31 days
82  days.append(30)   # Jun 30 days
83  days.append(31)   # Jul 31 days
84  days.append(31)   # Aug 31 days
85  days.append(30)   # Sep 30 days
86  days.append(31)   # Oct 31 days
87  days.append(30)   # Nov 30 days
88  days.append(31)   # Dec 30 days
89
90  mon = []
91  x = 0
92  for a in days:
93      x = x + a
94      mon.append(x)    # here, we create a list
95
```


First, we need to prepare the list
of the days for the start of each month.

The resulting list is:

	0	}	January
	31	}	
February {	59	}	March
	90		
	120		
	151		
	181		
	212		
	243		
	273		
	304		
	334		
	365		

Remember the slice function for lists (and strings).

Slice: extract text by position

b[0:2] or b[:2] → returns text from position 0 up to 2 (but not including 2)
b[2:] → returns text from position 2 to end

For the month of March we have to extract [2:3]
(position 2 and 3 in the list of months)

Now, we add this below our list of months.

```
96     ### create example output
97     x = range(1,366,1) # number of days from
98
99     m = 2 # choose one month
100    a = mon[m]          # this marks the first da
101    b = mon[m+1]        # this marks the first da
102    dm = x[a:b]         ← only days for the month
103
104    pngfile = 'Energy-' + str(m+1) + '.png'
```

list of days

We make a simple trick for the file name. Strings can be added.
Therefore, we include the number of the month in the name.

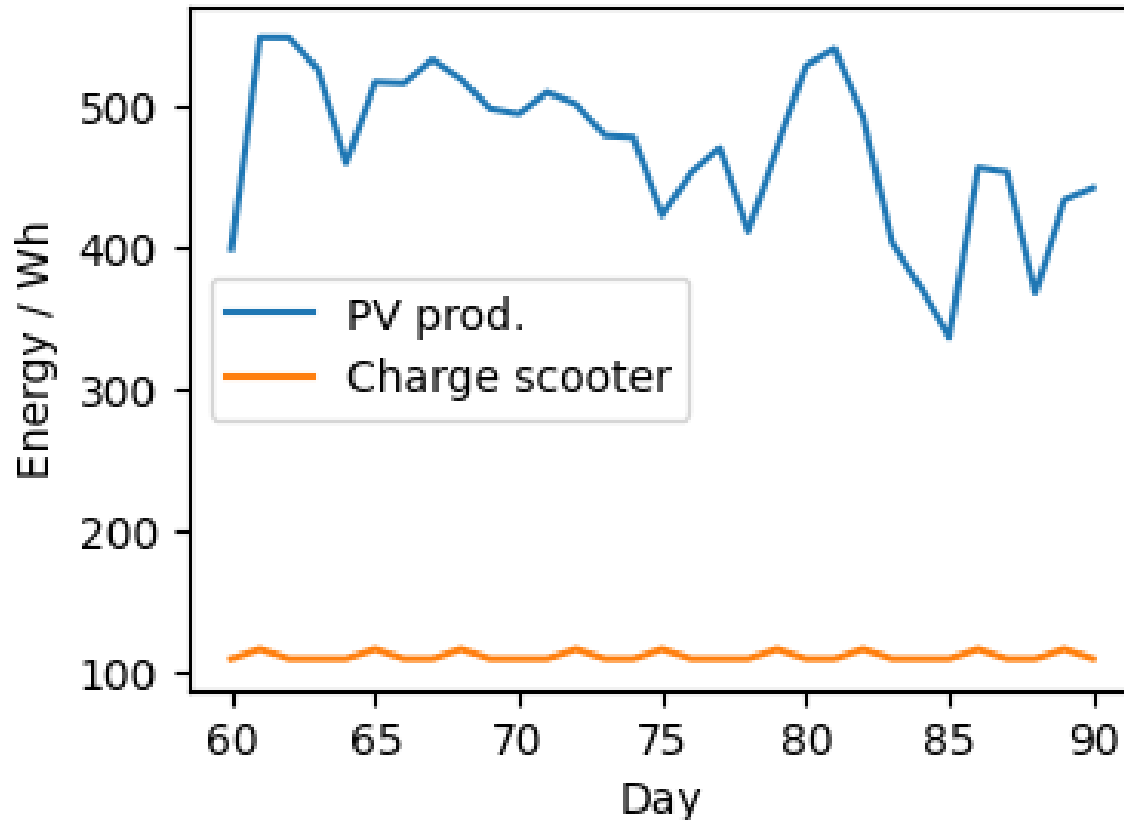
Now, we need to modify our plot in order to show only the y-value for the selected month.

```
109 plt.subplots( figsize=( fwidth,fheight ) )
110
111 y = pvpro[a:b]
112 plt.plot(dm, y, label='PV prod.') # line
113 y = chargesco[a:b]
114 plt.plot(dm, y, label='Charge scooter') #
115
```



list of days for the month

list of results for the month



These are the results for Kochi / for the month of March.

For an overview, it would be great to generate results for all months of the year.

Therefore, we need to create a loop.

```
96   ### create output for one year
97   x = range(1,366,1) # number of days from 1
98
99   m = 0
100  while m < 12:      # loop over all months
101
102       a = mon[m]      # this marks the first
103       b = mon[m+1]    # this marks the first
104       dm = x[a:b]     # cut out the days for
105       m = m + 1
106
107   pngfile = 'Energy-' + str(m) + '.png'
```

loop over all months

changed number of month

At the end, we need to close our figure, because we create a larger number of plots in the loop.

```
plt.savefig(pngfile  
plt.close()
```

We created a list of files:

Energy-1.png
Energy-2.png
Energy-3.png
Energy-4.png
Energy-5.png
Energy-6.png
Energy-7.png
Energy-8.png
Energy-9.png
Energy-10.png
Energy-11.png
Energy-12.png

You can click through each of those,
but it would be better to have an overview.

We can create a simple HTML page, which
shows all results in a table.

You can download the page from the Moodle system.

It is very simple to create such a page. Open the file 'results-energy.html' in your text editor.

```
<table>
```

```
<tr>
```

```
<td>  </td>
```

```
<td>  </td>
```

```
<td>  </td>
```

```
<td>  </td>
```

```
</tr>
```

```
<tr>
```

```
<th>January</th>
```

```
<th>February</th>
```

```
<th>March</th>
```

```
<th>April</th>
```

```
</tr>
```

```
</table>
```

In HTML, everything is enclosed in tags.

Example, this is an empty table:

```
<table> </table>
```

This is one row of a table:

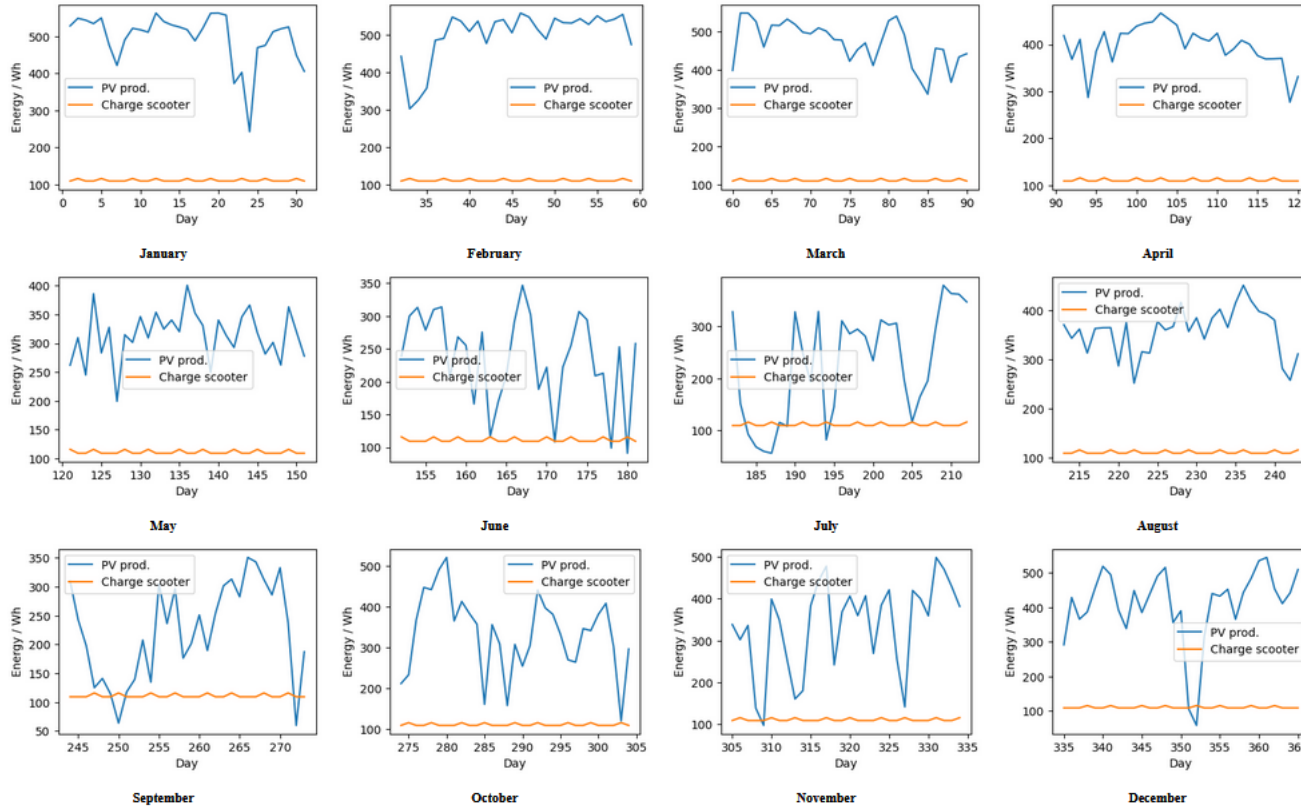
```
<tr> </tr>
```

This is displayed as header:

```
<th> </th>
```

These are the Results for the Project 'Scooter'.

* Energy production and consumption *



These are the results for the city of Kochi.

PV production and
energy consumption of the scooter

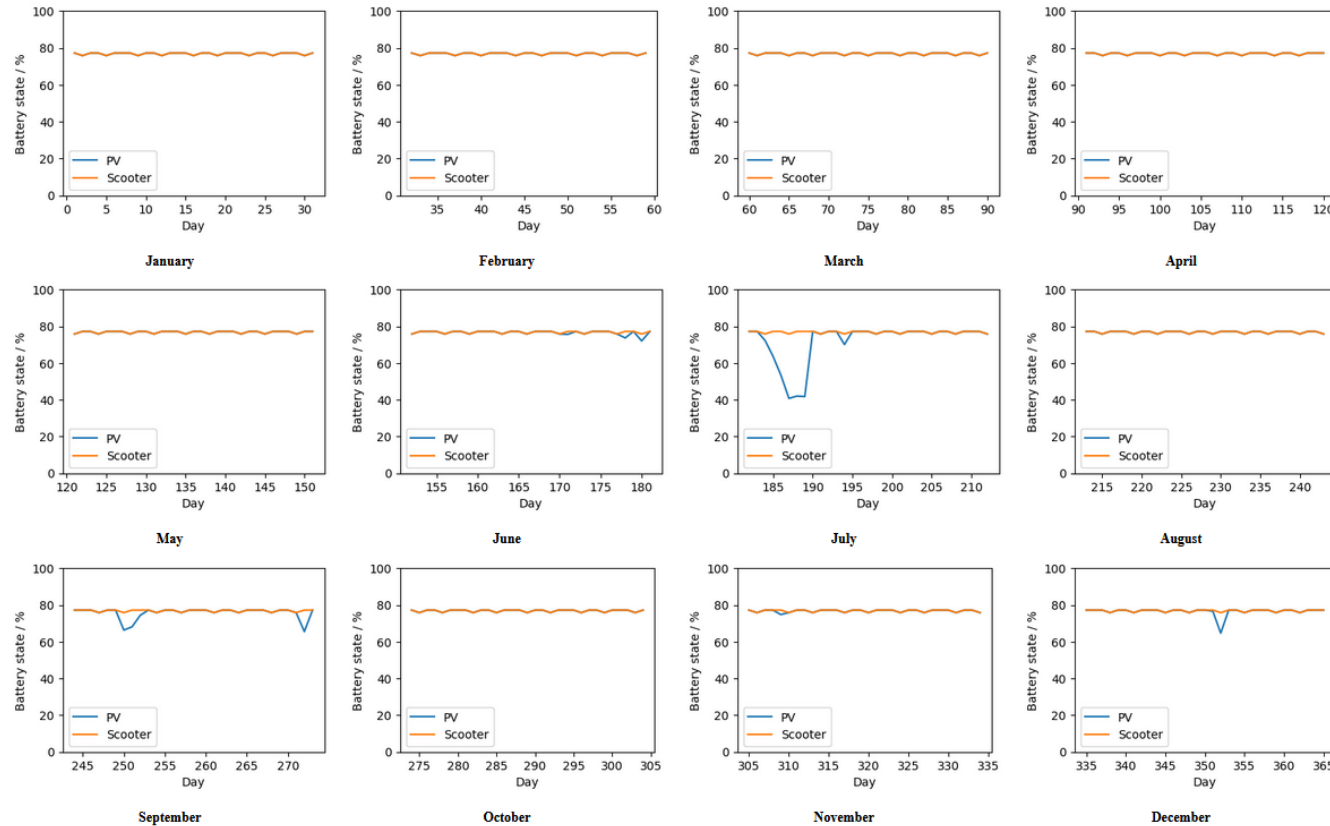
Let us create the same output for the
state of batteries.

Basically, we can almost copy the plotting
part for the energy.


```
127     ### Here, we add a second plot for the stat
128     pngfile = 'Battery-' + str(m) + '.png'
129     plt.subplots( figsize=( fwidth,fheight ) )
130
131     y = statepv[a:b]
132     plt.plot(dm, y, label='PV') # line plot
133     y = statesco[a:b]
134     plt.plot(dm, y, label='Scooter') # line
135
136     plt.xlabel('Day') # label x-axis
137     plt.ylabel('Battery state / %') # label
138
139     plt.ylim(0,100) ← here, we can easily fix the scale
140     plt.legend(loc='lower left')
141     plt.tight_layout() # optimize plac
142     plt.savefig(pngfile) # create file i
143     plt.close()
```

These are the results for the Project 'Scooter'.

* State of battery *



Results are displayed in the HTML page 'results-battery.html'.

You can see, it looks much better if all scales (y-axis) are identical and the legend is always placed at the same location.

As a last step, let's clean up the energy page with identical scales. Think about, what would be a reasonable upper limit for the energy scale?

Think about, what would be a reasonable upper limit for the energy scale?

Solution:

A good approach would be to take the peak power of the PV system and assume 8 hours of full power.

```
119 plt.xlabel('Day')          # label x-axis
120 plt.ylabel('Energy / Wh')  # label y-axis
121
122 z = sizepv * 1000 # conversion from kW to W
123 z = z * 8         # maximum value assumed full
124 plt.ylim(0,z)
```

We created a system model for the application of solar power to operate a scooter.

The following results can be displayed on a HTML page:

- PV production per day
- Charge to scooter
- State of battery solar (min value)
- State battery scooter (min value)

Your job: Write down the software documentation.

We have achieved Deliverable D5: system model!

